Controlling the Franka FR3 with ROS 2

A from-scratch guide to the ROS stack, CRISP controllers, the gripper, and how your TA-CBF policy drives the real robot

Workspace: `~/franka_ros2_ws` · Robot: FR3 @ 192.168.1.17 · Written for someone new to ROS

What this document is

You have a robot arm that already moves when you run example scripts. This guide explains **what is actually happening underneath** — every important program (node), every channel they talk over (topic / service / action), and how a line like `robot.set_target(...)` in Python ends up as motion of a real motor. No prior ROS knowledge assumed. Read Part 1 once; keep Parts 5–7 as a lookup reference.

Contents

1 · ROS 2 in ten minutes

ROS 2 (Robot Operating System 2) is not an operating system — it's a way to split robot software into many small programs that talk to each other over a network. The whole point: the camera program, the motor program, and your policy program don't need to know about each other's code — they just exchange *messages*.

The four words you must know

Concept	Analogy	What it is
Node	One employee	A single running program. `ros2_control_node` , `franka_gripper` , `rviz2` , and your Python script are each a node.
Topic PUB SUB	A radio channel / notice board	A named, typed stream of messages. Anyone can **publish** (broadcast) to it; anyone can **subscribe** (listen). Continuous, one-way, fire-and-forget. E.g. `/joint_states` streams the joint angles ~1000×/s whether or not anyone is listening.
Service SRV	A phone call	Request → wait → single response. Used for one-off commands, e.g. "switch to the impedance controller". Blocking, one-to-one.
Action ACT	Ordering a package	A long-running goal with *feedback* while it runs and a *result* at the end, and you can cancel it. The gripper uses actions: "grasp to 1 cm at 30 N" takes a second and reports success.

> **Publish / subscribe is the heartbeat of ROS.** 90% of "how does the robot work" is just: *which node publishes which topic, and which node subscribes to it.* Parts 4–5 are exactly that map for your robot.

Messages have types

Every topic carries one fixed **message type** — a struct with named fields. You'll meet these:

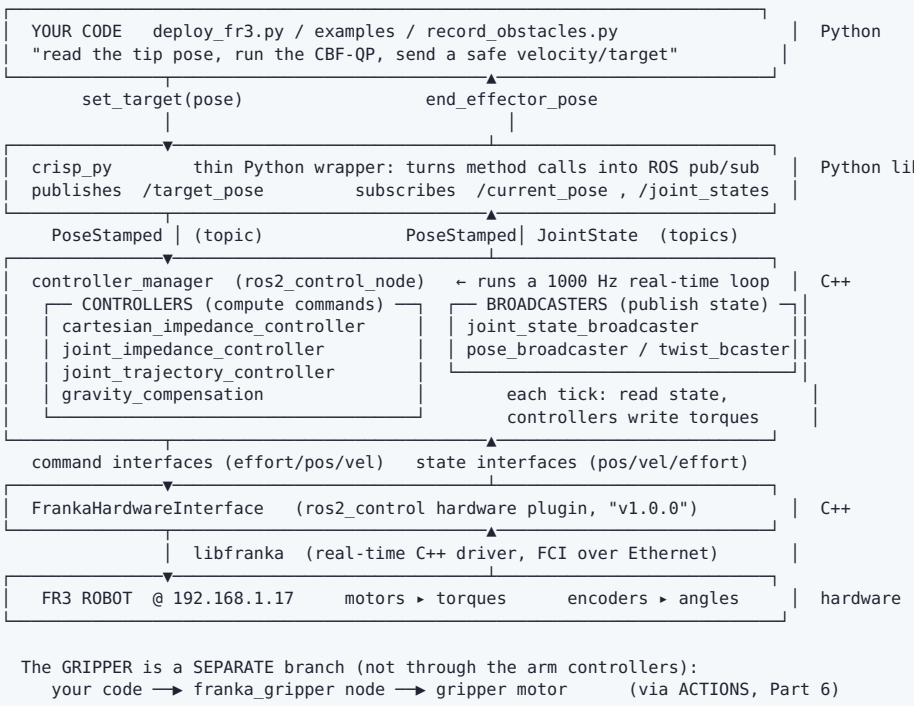
Message type	Fields you care about
`sensor_msgs/JointState`	`name[]` , `position[]` (rad), `velocity[]` (rad/s), `effort[]` (Nm) — one entry per joint.
`geometry_msgs/PoseStamped`	`pose.position` {x,y,z} in metres + `pose.orientation` {x,y,z,w} quaternion, in a named frame.
`geometry_msgs/TwistStamped`	Linear + angular velocity of the end-effector.
`franka_msgs/action/Grasp` , `/Move` , `/Homing`	Gripper goals: width, speed, force, epsilon tolerances.

Frames & TF

Every position is expressed relative to a **frame** (a coordinate system). Your world is anchored at `base` (the robot's foot). The needle-tip pose you care about is `fr3_hand_tcp`. ROS keeps a live tree of transforms (**TF**) between all frames so any node can convert between them. **Your TA-CBF config is already written in the `base` frame in metres** — the same frame the robot reports poses in.

2 · The layered stack: from your Python down to a motor

Reading top→bottom is the path a command travels. Bottom→top is the path sensor data travels.



The one idea to keep: your Python never sends torques. It publishes a *goal* (a target pose). A controller inside `controller_manager`, looping at 1 kHz, continuously compares that goal to the measured pose and computes the torques. That control loop is what makes the arm "soft" and compliant.

3 · `ros2_control`: how torques are actually produced

`ros2_control` is the framework in the middle layer. Three pieces:

- **Hardware interface** (`FrankaHardwareInterface`) — the adapter to the real robot. It exposes **state interfaces** (what you can read: each joint's position/velocity/effort) and **command interfaces** (what you can write: per-joint effort/position/velocity, plus Cartesian velocity/pose). Your launch log printed every one of these as it registered them.
- **controller_manager** — the conductor. Loads/activates controllers and runs the **1000 Hz** loop: *read all state interfaces → let the active controller compute → write command interfaces*.
- **Controllers** — plug-in strategies. Two flavours:
  - **Controllers** that *write* commands (produce motion): impedance, trajectory, gravity-comp.
  - **Broadcasters** that only *read* and *publish* state to topics (produce no motion): `joint_state_broadcaster`, `pose_broadcaster`.

Only **one** motion-producing controller should be active on the same joints at once (otherwise two strategies fight over the torques). Broadcasters are always safe to run alongside. You switch the active controller with a **service** (Part 8), or crisp_py's `switch_controller(...)`.

CRISP = the controllers, specifically

CRISP ("Compliant ROS 2 controllers") is just a package of high-quality `ros2_control` controllers tuned for torque-based, compliant manipulation. `cartesian_impedance_controller` is the star: you give it a *target pose*, it behaves like a virtual spring pulling the end-effector there — compliant, so it won't fight you or the tissue. That is exactly the low-level controller your TA-CBF policy sits on top of.

4 · The controllers running on YOUR robot

These loaded when you ran the CRISP `franka.launch.py`. "Active" = currently steering; "loaded/inactive" = ready but not steering; activate it when you want it.

Controller	Kind	State	What it does
`joint_state_broadcaster`	broadcaster	active	Publishes all joint positions/velocities to `/joint_states`. Pure sensor output.
`pose_broadcaster`	broadcaster	active	Computes the end-effector pose and publishes it (this is your `/current_pose` source).
`twist_broadcaster`	broadcaster	active	Publishes end-effector linear/angular velocity.
`joint_trajectory_controller`	controller	active	Holds / moves the joints along a trajectory (effort-based). On startup it just holds the current pose.
`cartesian_impedance_controller`	controller	loaded	**The main one.** Subscribe a target pose → compliant spring-like motion to it. Activate to move the arm by pose. This is what TA-CBF drives.
`joint_impedance_controller`	controller	loaded	Compliant control in joint space instead of Cartesian.
`gravity_compensation`	controller	loaded	Cancels gravity so you can **hand-guide** the arm freely. Activate this for recording demos / tracing obstacles.
`torque_feedback_controller`, `external_torques_broadcaster`	—	fail	**Harmless.** They error with "'*type*' *param not defined*" — a demo-config gap for these two optional extras. Everything you use is unaffected. Ignore.

5 · The topic map (who publishes, who listens)

This is the pub/sub picture of your running system — the single most useful reference here.

Topic	Type	Dir	Meaning / who
`/joint_states`	sensor_msgs/JointState	**PUB** by broadcaster	Live joint angles/velocities @~1 kHz. RViz & crisp_py subscribe.
`/current_pose`	geometry_msgs/ PoseStamped	**PUB** by pose_broadcaster	Live end-effector pose in `base` frame. **crisp_py `robot.end_effector_pose` reads this.**
`/current_twist`	geometry_msgs/ TwistStamped	**PUB**	Live EE velocity.
`/target_pose`	geometry_msgs/ PoseStamped	**SUB** by cartesian ctrl	The goal pose you command. **crisp_py `robot.set_target(...)` publishes this.** The impedance controller tracks it.
`/target_stiffness`, `/target_wrench`	std/geometry msgs	**SUB**	Optional: vary spring stiffness / apply a force online.
`/franka_gripper/joint_states`	sensor_msgs/JointState	**PUB** by franka_gripper	Finger positions; `width = position[0]+position[1]`.
`/gripper/joint_states`	sensor_msgs/JointState	**PUB** by adapter	Normalised (0–1) width the crisp_py gripper reads (Part 6).
`/gripper/ gripper_position_controller/ commands`	std_msgs/Float64MultiArray	**SUB** by adapter	Normalised gripper command crisp_py publishes; the adapter turns it into a franka_gripper action.
`/robot_description`	std_msgs/String	**PUB** by robot_state_publisher	The URDF (robot model) as text. RViz & controllers read it to know the kinematics.
`/tf`, `/tf_static`	tf2_msgs/TFMessage	**PUB**	Live transforms between all frames. RViz uses them to draw the arm.

Gripper actions (not topics)

Action	Type	Purpose
`/franka_gripper/homing`	franka_msgs/Homing	**Calibrate** the fingers (open→close to find limits). **Required once after power-up** or move/grasp silently no-op.
`/franka_gripper/move`	franka_msgs/Move	Open/close to a target width at a speed (no clamping force).
`/franka_gripper/grasp`	franka_msgs/Grasp	Close to a width **and hold with a force** — use for gripping the needle.

6 · The gripper: actions, the bridge, and the bug you hit

The Franka Hand is driven by its own node, `franka_gripper`, over the three **actions** above. That's the reliable interface. There is *also* a convenience path through crisp_py — and that's where your "nothing happens" came from.

Two ways to command the gripper

```
A) DIRECT  (reliable, full control of width + force)          ← recommended
   your code ──ACTION──► /franka_gripper/{homing,move,grasp} ──► franka_gripper ──► motor

B) VIA crisp_py  (a fragile toggle bridge — the source of your bug)
   gripper.open()/close()
      └─PUB Float64─► /gripper/gripper_position_controller/commands
                        └─► crisp_py_franka_hand_adapter ──ACTION──► /franka_gripper/grasp|homing
```

Why `06_franka_gripper.py` did nothing. Path B goes through `crisp_py_franka_hand_adapter`, which only *toggles*: it closes *only if* it currently thinks the gripper is open, and opens *only if* it previously closed (an internal `is_closing` flag). After you'd already moved the fingers with direct `move` / `grasp` tests, that flag and the real finger width were out of sync, so the adapter judged the command redundant and issued nothing. It's a known-fragile demo bridge, not a fault in your setup.

Do this instead. Use path A directly (your `gripper_manual.py`, or `ros2 action send_goal`), and **home first**:

```
# 1) HOME (fingers fully open then close — keep clear). Required after power-up.
ros2 action send_goal /franka_gripper/homing franka_msgs/action/Homing "{}"

# 2) OPEN to 6 cm
ros2 action send_goal /franka_gripper/move franka_msgs/action/Move \
  "{width: 0.06, speed: 0.1}"

# 3) GRASP a thin needle: close to ~1 mm, hold 30 N, wide tolerance so it still 'succeeds'
ros2 action send_goal /franka_gripper/grasp franka_msgs/action/Grasp \
  "{width: 0.001, speed: 0.05, force: 30.0, epsilon: {inner: 0.005, outer: 0.005}}"
```

If step 2/3 still no-op, you skipped homing — that's the #1 cause.

7 · crisp_py: how the Python wrapper maps to ROS

crisp_py exists so you don't hand-write publishers/subscribers. Each convenience call is really a pub or sub:

Python (crisp_py)	What it does in ROS
`robot = make_robot("fr3")`	Starts a node and a background thread that *spins* it (processes incoming messages). Without spin, nothing arrives — that's why it runs in a thread.
`robot.wait_until_ready()`	Blocks until `/current_pose` **and** `/joint_states` are arriving. Its timeout is the "robot not available" error you saw when the launch wasn't up.
`robot.end_effector_pose`	**SUB** latest `/current_pose` → returns a `Pose` with `.position` = [x,y,z].
`robot.set_target(position=[x,y,z])`	**PUB** a `PoseStamped` on `/target_pose`. The active Cartesian controller tracks it.
`robot.controller_switcher_client.switch_controller("cartesian_impedance_controller")`	**SRV** call to controller_manager to make that controller the active one.
`make_gripper("gripper_franka")`	Node for path B above (the fragile bridge). Prefer direct actions for reliability.

Where the source lives. You `pip install`ed crisp_py, so the *running* copy is inside the venv at `…/venv/lib/python3.10/site-packages/crisp_py/…`, not the repo under `src/crisp_py/`. Edit the repo copy for reading; re-run `pip install` (or install `-e`) to make edits take effect.

8 · How YOU control the bot — the concrete recipes

Every session is two steps: (1) bring the stack up, (2) command it.

Step 1 — bring up the stack (Terminal 1, leave running)

```
source /opt/ros/humble/setup.bash
source ~/franka_ros2_ws/install/setup.bash
ros2 launch crisp_controllers_robot_demos franka.launch.py \
    robot_ip:=192.168.1.17 arm_id:=fr3 load_gripper:=true use_rviz:=true
# brings up: hardware iface + controller_manager + all controllers/broadcasters
#           + franka_gripper + RViz (live mirror of the real arm)
```

Pre-reqs in Franka Desk (https://192.168.1.17/desk): joints **unlocked**, **FCI activated**, user-stop released.

Step 2 — command it (Terminal 2)

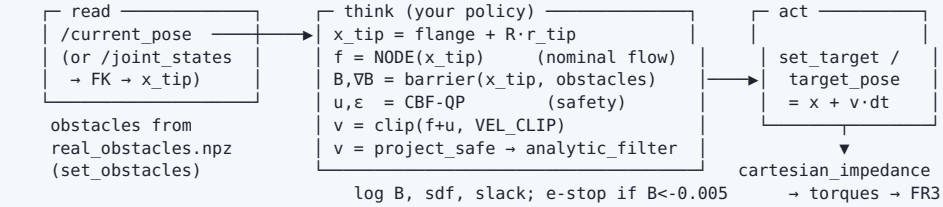
```
source /opt/ros/humble/setup.bash
source ~/franka_ros2_ws/install/setup.bash
source "~/franka_ros2_ws/src/Tolerance_Aware_CBF(TA-CBF)/venv/bin/activate"

# activate the compliant Cartesian controller, then send pose targets
ros2 control set_controller_state cartesian_impedance_controller active
cd ~/franka_ros2_ws/src/crisp_py
python3 examples/01_figure_eight.py       # moves via /target_pose
```

> **Safety:** first hardware runs — keep a hand on the e-stop, use small motions, and for TA-CBF start with `VEL_CLIP = 0.02 m/s`. Activate only **one** motion controller at a time.

9 · Your TA-CBF deployment data-flow

Putting it together — this is what the deploy node will do every 10 ms (100 Hz):



So: **subscribe** the pose, run your CLF-CBF-QP, **publish** a safe target. The compliant controller does the rest. The obstacles come from the clouds you record with `record_obstacles.py`; the map/workspace is fixed in `config.py` (already in `base`-frame metres).

10 · ros2 command-line cheat sheet + glossary

Inspect anything that's running

```
# what nodes / topics / actions exist right now
ros2 node list
ros2 topic list                        ros2 action list

# SEE a topic's data live, and how fast it updates
ros2 topic echo /current_pose          ros2 topic hz /joint_states
ros2 topic info /target_pose -v        # type + who pubs/subs

# the controllers and their state (active / inactive)
ros2 control list_controllers
ros2 control set_controller_state cartesian_impedance_controller active

# call a gripper action by hand
ros2 action send_goal /franka_gripper/homing franka_msgs/action/Homing "{}"

# who is a node, what does it pub/sub?
ros2 node info /ros2_control_node
```

Glossary

FCI	Franka Control Interface — the real-time channel libfranka uses to talk to the robot. Enable it in Desk.
URDF	The robot's geometry/kinematics model (published on `/robot_description`).
Broadcaster	A controller that only reads state and publishes it — never moves the robot.
Impedance control	Torque strategy that behaves like a spring+damper toward a target pose — compliant, safe against contact.
Spin	Letting a node process its incoming/outgoing messages. crisp_py spins on a background thread for you.
Namespace	A prefix that groups a node's topics (e.g. `/NS_1/...`). Your CRISP demo runs without one.
TF	The live tree of coordinate-frame transforms.

Generated for `~/franka_ros2_ws` · FR3 + franka_ros2 (Humble) + CRISP + crisp_py · reflects the controllers, topics, and actions observed on your running system. Names verified against your configs; if you add a namespace or switch controllers, re-check with `ros2 topic list` / `ros2 control list_controllers`.